Engineering Requirements for Generative AI Agents: The BARD Process Theory

Alfonso Cannavale*, Antonio Della Porta*, Gianmario Voria*, Gilberto Recupito†,
Maria Teresa Baldassarre‡, Gemma Catolino*, Fabio Palomba*, Andrea De Lucia*, Stefano Lambiase§
*University of Salerno, Italy †Vrije Universiteit Brussel, Belgium
‡University of Bari, Italy §Aalborg University, Denmark

***Abstract*—How teams engineer requirements for systems built around Generative AI (GenAI) Agents remains empirically uncharted. Existing Requirements Engineering (RE) research mainly addresses AI and ML-based systems broadly but provides no process account for agentic systems, whose non-deterministic behavior cannot be anticipated at design time. To understand how teams actually proceed, we applied Socio-Technical Grounded Theory across 36 semi-structured interviews with practitioners, interleaving data collection and analysis so that emerging findings drove our theoretical sampling and shaped successive rounds of coding. The result is Behavioral Agent Requirement Discovery (BARD), the process theory of RE for GenAI Agents. BARD holds that requirements are not fixed up front but discovered through a recurring cycle of four activities: teams scope the need, build the agent, observe its behavior, and revise what they ask for. Through this four-phase loop, the requirement evolves from a fixed specification towards behavioral and evaluation criteria. Four tensions drive transitions between phases, and two conditions frame the loop. BARD provides researchers with a framework for iteratively refining RE workflows, while giving practitioners a structured basis to embed into their development processes.**

***Index Terms*—Requirements Engineering, Generative AI, Agents, Grounded Theory, Empirical Software Engineering.**

I. Introduction

Generative AI (GenAI) Agents are autonomous software systems that combine Large Language Model (LLM) reasoning with tool use to act on behalf of a user [1]–[3]. Within a few years, they have moved from research prototypes to industrial deployment with recurring examples where workflow assistants chain multiple steps to complete a business task, coding assistants read and modify entire repositories, and customer-facing assistants look up information and update records on behalf of a user. What distinguishes these systems from earlier AI-powered software is a combination of three properties: a non-deterministic reasoning core [4], the ability to invoke external tools mid-execution [5], [6], and a degree of autonomy in deciding when and how to act. Engineering software around such a core raises questions that established practices do not yet routinely answer [7], [8].

Over the years, Requirements Engineering (RE) has shown its relevance for the definition of complex systems. As a mature discipline, RE practices remain a key pillar in the AI context, enabling the modeling and development of AI-enabled systems. However, the shift towards these systems has changed the characteristics of this process, introducing new roles, activities, and practices [8]. With the introduction of agents, this evolution accelerates further, requiring RE practices to account for systems whose behavior cannot be fully anticipated at design time.

This evolving relationship between RE and AI has itself become subject of study: RE for AI- and ML-based systems has grown into an active research area in recent years. Villamizar et al. [9] surveyed RE practices for ML systems, and Habiba et al. [8] extended the lens to AI-based systems more broadly across 126 primary studies. Workshop papers in the IEEE RE conference series [10], [11], together with the framing piece of Dalpiaz and Niu [12], set the early agenda. Wan et al. [13] combined interviews with a follow-up survey to characterize how ML changes development practice. None of this work, however, tried to examine the RE *process* through which requirements for GenAI Agent systems are actually elicited, negotiated, refined, and validated in practice. As a result, **current knowledge remains fragmented** across narrower concerns, such as non-functional requirements, evaluation, prompt design, and quality assurance, without a shared process frame that explains how these concerns are connected during development. This fragmentation limits cumulative understanding: researchers lack a common reference against which to position empirical findings, educators and tool builders lack a process-oriented anchor for curricula and design decisions, and practitioners are left to rely on local heuristics rather than a shared vocabulary for comparing and improving their practices.

More broadly, this gap reflects the **limited availability of substantive theory** in socio-technical software engineering (SE) research [14], [15]. Substantive theories, and process theories in particular, play an important integrating role in mature empirical disciplines by explaining how phenomena unfold over time and how their constituent activities relate to one another [16]. This need has motivated theory-building approaches in SE, such as Socio-Technical Grounded Theory [15]. In the absence of such a foundation for GenAI Agent RE, the field risks accumulating disconnected reports on individual practices, while teams continue to rediscover similar trade-offs in isolation.

To address these gaps, we developed the *substantive process theory* [15]–[17] of how practitioners engineer requirements

for GenAI Agent systems.[1]

We then formulated the following research question:

> **RQ**—*How do practitioners engineer requirements for software systems built around GenAI Agents?*

To answer the question, we applied Socio-Technical Grounded Theory (STGT) [15] to develop a substantive theory; we interviewed 36 practitioners recruited by convenience sampling, snowballing, and self-selection (integrating screening criteria). As a result, we provide the following contributions: (1) *Behavioral Agent Requirement Discovery* (BARD), i.e., to the best of our knowledge, the first substantive theory of the RE process for GenAI Agents; (2) an empirically grounded characterization of the requirements, artifacts, and decisions agent teams reason about—e.g., system prompts and evaluation suites; and (3) implications for researchers and practitioners: the former obtain a shared frame in which to position narrower results and sharpened questions for follow-up research, the latter a vocabulary and process anchors.

II. Background and Related Work

This section reports background and related work. It is important to note that, as is common in GT studies, we deliberately keep the review limited to avoid biasing the coding process, adhering to the principle of theoretical sensitivity and non-committal literature review [15], [17]. The focus here is on justifying the research gap and on verifying novelty [15]. The Discussion (Section V) compares our results with existing findings and engages the broader body of knowledge.

A. Generative AI Agents

The role of GenAI and LLMs in SE has been the object of research in previous years [3], [18]–[24]; in this paper, our focus is not on the use in software development, rather on the process to develop such systems.

We use the term *GenAI Agent* to denote software systems that combine LLM reasoning with tool use to pursue goals delegated by a user. Sapkota et al. [1] distinguish between *AI Agents* (single-agent systems oriented to specific tasks) and *Agentic AI* (multi-agent or strongly autonomous configurations); our investigation covers both ends of this distinction, because the engineering decisions practitioners face span single-agent integrations as well as multi-agent orchestrations. Furthermore, Wang et al. [2] provide a broader survey of LLM-based autonomous agents that situates the technical primitives we discuss.

B. Requirements Engineering for AI and ML Systems

Requirements engineering for AI- and ML-based systems has grown into an active subfield. Two systematic mappings cover its scope. Villamizar et al. [9] focus on machine-learning systems, while Habiba et al. [8] cover AI-based systems more broadly across 126 primary studies. The wider engineering perspective is set by the survey by Martínez-Fernández et al. [25] of 248 SE-for-AI primary studies and by Ahmad et al. [26], mapping 43 primary studies in RE-for-AI. Similarly to the motivation of this study, both studies highlight that RE strategies and tools are critical for AI-enabled systems because of the fundamental shift in the nature of these software systems. Indeed, early research agendas are anchored in Requirements Engineering Conference Workshops (REW) papers by Vogelsang and Borg [10] and Belani et al. [11], and in the framing piece of Dalpiaz and Niu [12].

The empirical study closest to ours in design is that of Wan et al. [13]. They combine 14 interviews with a follow-up survey of 342 respondents across 26 countries to characterize how machine learning changes development practice, and report systematic differences between ML-based and traditional software development across the lifecycle—in requirements work, data management, testing, and deployment. However, our work differs in object and scope. Wan et al. study how a single technological shift (machine learning) reshapes development practices broadly, while we focus on the workflows and activities that shape the RE process for systems built around GenAI Agents specifically.

C. Agent-Oriented Requirements Engineering (Pre-LLM)

SE has a long tradition of agent-oriented RE that predates large language models. Its two pillars are KAOS, surveyed by van Lamsweerde [27], and Tropos, introduced by Bresciani et al. [28]; the broader landscape of goal-oriented RE is documented by Horkoff et al. [29]. These traditions assume rational agents that pursue explicit goals through deliberate plans, with the reasoning logic hand-crafted by the engineer. GenAI Agents present a different object: the reasoning core is a stochastic LLM, behavior is shaped by prompts and tool descriptions rather than by goal hierarchies, and autonomy is exercised over runtime tool selection rather than over plan synthesis. We keep the vocabulary of *agent* but treat the engineering problem as new.

> **Background: Summary and Research Gap.**
>
> Three gaps emerged from the related works. First, no empirical study targets the RE process for GenAI Agent systems specifically. Second, work in the adjacent literature addresses narrower facets—e.g., non-functional requirements for ML systems, perception-system requirements, and broad ML development practices—rather than the RE process for agentic systems as a whole. Third, no STGT or comparable qualitative theorization has been applied to this object.

III. Research Method

Our methodology (depicted in Figure 1) follows the complete STGT [15], [30], a method that is increasingly adopted in SE [31]–[36]. Following the ACM/SIGSOFT Empirical Standards [37], we organized our reporting according to the *General Standard* and the *Grounded Theory* guidelines.

[1] A *substantive* theory is developed for a specific area of inquiry (in this case, RE) and is bound to its empirical context [17].

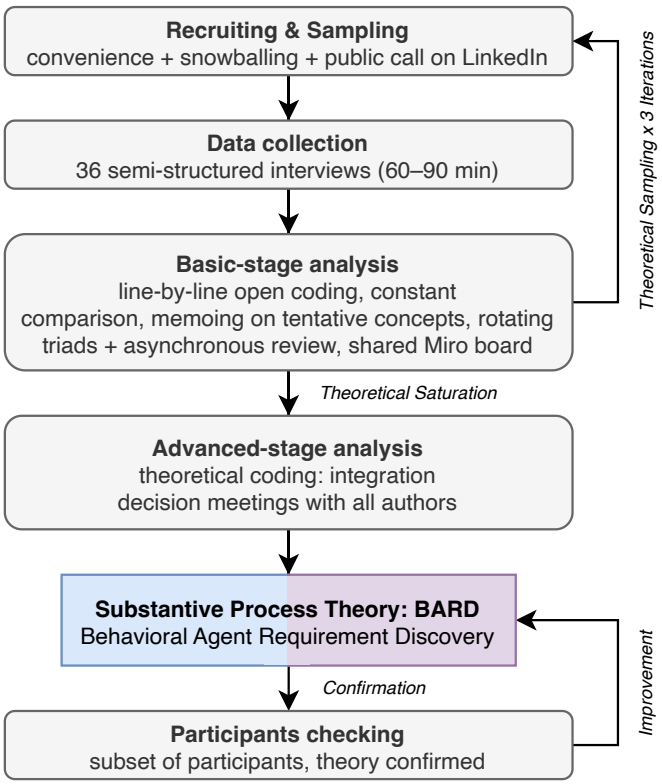


Fig. 1. Research Method.

We adopted STGT, rather than other variants [38]–[40], for two reasons: first, the object of study—as well as software development in general—is *de facto* socio-technical: software systems are built by teams within organizational workflows, and STGT is explicitly designed for this kind of object. Second, the question we pursue is a process question (*how* practitioners engineer requirements), for which existing theory is absent. Iterative data collection and theoretically grounded sampling that STGT includes fit the immaturity of the field, where the relevant categories are not yet known.[2]

We conducted *iterative semi-structured interviews*, in which we iterated over recruitment, data collection, and basic-stage coding until saturation. Subsequently, advanced-stage analyses have been performed to link concepts and build categories. Once the theory was built, we contacted the participants who had opted in at the end of their interviews to share the resulting theory and gather their feedback, asking them: (1) whether they saw their own experiences and processes reflected in the theory; and (2) whether they had any concerns about its structure or overall content.

In our analysis, we adopted an interpretivist epistemological stance, i.e., practitioners' accounts were treated as situated knowledge to extract a unified ground from their experience, rather than objective observations of an external phenomenon [37].

[2]Ethical approval was obtained from Aalborg University.

A. Participants and Sample Diversity

Participants are practitioners actively involved in the design, integration, or development of GenAI Agent systems. We required involvement in either the elicitation phase (scoping what the agent should do) or the analysis phase (specifying functional and non-functional requirements); roles not actively involved in requirement engineering activities were excluded.

Demographics such as role, development context, country of work, and years of experience for each participant were tracked. Diversity was actively pursued along four dimensions: development context (to avoid over-representation of any single industry), organizational type (consultancy, product company, in-house engineering, agency), country of work (to surface variation in regulatory and organizational context), and seniority. This deliberate spread addresses the GT antipattern, flagged in the SIGSOFT Empirical Standards [37], of small homogeneous samples that create the illusion of theoretical saturation.

We adopted convenience sampling [41], complemented by snowball sampling (via referrals from early participants) and self-selection to recruit participants. We started from our professional network and from contacts at consultancy and product companies (anonymized), and extended coverage through a public LinkedIn post inviting expressions of interest. Theoretical sampling was operationalized by examining which categories remained under-saturated after each iteration and using those gaps to guide subsequent recruitment toward practitioners likely to offer complementary perspectives.

In terms of demographics, as reported in Table I, our sample includes 36 practitioners with experience developing GenAI agents. To contextualize our sample, we compared its profile with the broader developer population reported in the Stack Overflow Developer Survey [42]. We found close correspondence in the share of freelancers and contractors (11.1% vs. about 14% self-employed) and in a role mix dominated by developers and architects. Our gender split (19.4% women) is more balanced than that survey's own respondent pool and closer to workforce-level estimates of women in SE (about 23–25%), while professional experience spans early-career to senior practitioners in comparable proportions. We read these correspondences as evidence of the sample's plausibility and transferability [41].

B. Data Collection

Interviews were semi-structured and lasted between 60 and 90 minutes, and we recorded them with the participant's consent and transcribed them for analysis.

Each interview was conducted by two to three researchers, so that the questioning never rested on a single interviewer. After a short introduction and the notices on personal-data handling and research ethics, the researchers took the participant through the protocol. The protocol was inspired, at a high level, by the traditional RE process—namely, scoping, analysis, and validation [43]—and it encouraged participants to use their own vocabulary to describe their work. The

TABLE I
SAMPLE PROFILE ($N = 36$).

Category	n	%
Gender		
Male	29	80.6%
Female	7	19.4%
Country of work		
Europe	22	61.1%
Canada	8	22.2%
United States	6	16.7%
Years of experience in software development or AI/ML		
1–3	12	33.3%
4–7	13	36.1%
>8	11	30.5%
Development context		
Company, external clients / consultancy	19	52.8%
Company, internal product	13	36.1%
Freelancer / contractor for clients	4	11.1%
Project role		
Software Developer / Engineer	26	72.2%
Product Owner / Product Manager	5	13.9%
Solution Architect / AI Architect	3	8.3%
Others	2	5.6%
Team size		
1–3	15	41.7%
4–8	11	30.6%
9–30	10	27.8%

participant information sheet, included in the supplementary material, details the protocol as administered.

Ethical approval for the study was obtained from the institutional review boards of all participating universities, and the specific procedures are reported in our appendix [44].

C. Data Analysis

Basic-stage analysis followed open coding line-by-line on the interview transcripts, with constant comparison within and across cases and memoing on tentative concepts (Figure 2 illustrates an example). Coding sessions were conducted in rotating groups of three authors at a time, while the remaining authors provided asynchronous feedback on the codes produced; this arrangement balanced in-session discussion with independent review across sessions. Moreover, all artifacts—i.e., codes, code groupings, memos, and tentative concept relationships—were maintained on a shared online board on Miro [45], which served both as the working coding surface and as the audit trail for memos and codebook versions described later in this subsection. To ensure traceability and verifiability, our coding pipelines are fully documented in our public replication package [44].

Theoretical sampling followed an analyze-then-recruit cycle: gaps and unsaturated properties identified during basic-stage coding directed the recruitment of subsequent interviewees, steering us toward practitioners whose profile (role, domain, organizational type) was likely to fill the remaining

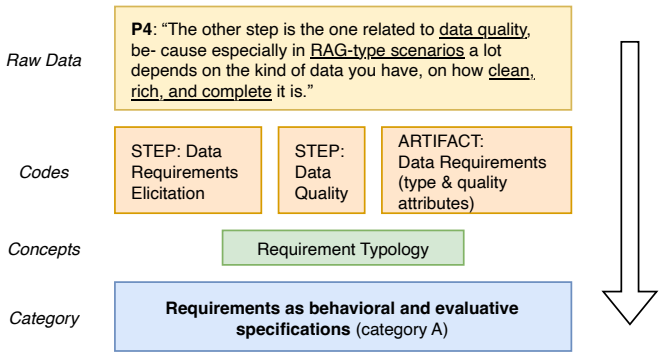


Fig. 2. Example of open coding and constant comparison.

gaps. We operationalize theoretical saturation as the absence of new properties or relationships in the last 11 interviews; in total, we conducted 3 iterations of data collection and analysis.

Memos were written throughout the analysis, ranging from open-coding memos on individual codes to concept memos on emerging categories. As described above, all memos and successive versions of the codebook—as well as decision rules—were maintained on the shared online board, which provides a versioned audit trail.

Collecting and analyzing the data needed to build the theory was a labor-intensive process. Excluding the interviews themselves, each researcher invested roughly 20 hours coding and a further 10 hours on theory-building, about 30 person-hours per analyst.

D. The Research Team and Reflexivity

Developing a solidly grounded theory in a socio-technical context requires a research team with a heterogeneous background [15], [31], [46]. Heterogeneity matters here because the topic spans RE, generative AI as a class of technical systems, and the socio-technical practice of teams that adopt it. The team brings together complementary research backgrounds along three axes: two senior researchers with prior experience conducting STGT studies; one researcher with broad expertise in qualitative methods, who guided the methodological process; and additional researchers with deeper expertise in the technical aspects of generative AI systems. All authors share a SE research background, which provides a common analytical vocabulary while leaving room for the disciplinary differences from which the analysis benefits.

Following Hoda's recommendation that GT work in SE be conducted as a collaborative analytic exercise rather than partitioned into individual roles [15], [30], all authors participated in both interview conduct and coding. Interviews were distributed across the team to balance the load and to expose participants to interlocutors with complementary backgrounds. Coding sessions, as described in Section III-C, were conducted in rotating groups of three so that every coding decision was discussed in real time before being committed to the shared board. This arrangement also served the reflexivity goal. By surfacing differences in interpretation as they arose, it ensured that no single author's lens dominated the analysis.

We treat reflexivity as a methodological obligation, not as a closing disclosure. We acknowledge three sources of bias that shaped—and that we worked to balance—the analysis. First, all authors are SE researchers, and our reading of practitioner accounts is shaped by the vocabulary internal to SE and by assumptions about what counts as good engineering practice. Second, the team holds divergent views on the actual usefulness and appropriate role of GenAI Agents in software production, ranging from cautious endorsement to skepticism; we deliberately preserved this internal disagreement through the coding process, treating it as useful input for the analysis rather than as noise to be averaged out. Third, the team is balanced between authors who lean toward more positivist and toward more interpretivist epistemological commitments.

Our principal mitigation strategy was consensus-building. Divergent codings and competing interpretations were discussed in coding meetings until either a shared interpretation was reached or the disagreement itself was logged as a memo for resolution at the advanced stage. We preferred consensus over mechanical inter-coder reliability metrics such as Cohen's κ, in line with Hoda's recommendation for STGT [15], because the goal of basic-stage coding in grounded theory is the development of a shared analytic vocabulary rather than the reproduction of pre-defined codes.

E. Trustworthiness of the Theory Building Process

We map our design to the qualitative trustworthiness criteria common in SE research, as discussed by Stol et al. [46]. *Credibility* is supported by using multiple coders working through consensus and by the depth of practitioner accounts elicited through a semi-structured interview protocol that allowed unexpected aspects to emerge. *Dependability* is supported by the audit trail recorded on the shared coding board (memo history and codebook versions). *Transferability* is supported by reporting sufficient context—namely, sample composition, organizational settings, and technical scope—for readers to assess applicability to their own setting.

Moreover, we seek *confirmation* of the resulting theory through participant feedback. We returned to a random subset of the interviewed practitioners (6 of them) with the integrated theory and gathered their feedback, in line with the GT-standard recommendation on member checking and dialogical interviewing; their feedback confirmed the theory as integrated, which we report in the next section. In general, participants reported that the theory was clear and well-presented from the very first version; minor changes were made to clarify the transition between the various macro-phases (see the next section for more details).

IV. BARD: A Substantive Process Theory of Requirements Engineering for GenAI Agents

This section presents the result of the socio-technical grounded theory process.

We first present the categories that emerged from our coding (Section IV-A), and then illustrate how they integrate into a unified theory (Section IV-B). To guide the reader visually, concept names and related constructs are underlined, ***category names*** appear in bold italics, and verbatim interview quotations are in *italics*. Notably, category names and final theoretical constructs do not correspond one-to-one. In GT, categories serve as the initial empirical building blocks, whereas the substantive theory recombines and abstracts them.

A. Constituent Categories

We identified seven categories, described below, followed by two cross-cutting dimensions.

A ***Requirements as behavioral and evaluative specifications*** are the requirements a team writes for an agent, and they differ from ordinary requirements in two ways. First, agents call for new kinds of requirements that traditional software does not have, as noted by P1, 💬 *"the possibility of deciding the degree of depth the LLM can adopt in its reasoning is something you can set as a requirement only in an agentic system."* Second, because a team often cannot ask for an exact output, the requirement becomes a target for behavior, and much of it stays implicit. P1 captured this directly, 💬 *"there is a requirement that is always both stated and unstated, that the model must work well."* (🧍 24 participants contributed to this).

B ***Iterative requirement discovery*** is how requirements take shape over time rather than being fixed at the start, since the team builds a first version, watches the agent, and revises the requirements accordingly. P2 described this, 💬 *"the process was iterative; in an initial phase we built a simple flow that was then adjusted on the basis of the agent's behavior."* New requirements also surface later, and P3 said, 💬 *"the requirements generally reach us already defined, though of course they always have to be redefined."* (🧍 24 participants contributed to this).

C ***The prompt as a requirement-implementation boundary object*** is about the system prompt, which serves as both a requirement and a piece of implementation, because the team writes it like code yet uses it to define what the agent will do. P2 captured this when they said, 💬 *"defining the system prompt is more of an implementation choice, but it should be done during requirements engineering."* Which side it falls on depends on the project, as P4 explained, 💬 *"if using AI is our own implementation choice, it is technical, but if the project starts from the client's need, the prompt is analyzed directly during requirements elicitation."* The prompt is continually refined by teams as testing reveals new needs, so that the requirement keeps changing within it—P3 noted, 💬 *"I often have to change old prompts because the requirement changes, since we test continuously… a kind of drift."* We call this continuous prompt refinement, and it is where the requirement's evolution leaves its trace on the artifact. (🧍 23 participants contributed to this).

D ***Capability-driven technology selection*** is the choice of model and tools, made as part of RE, where the requirement names what the model must be able to do rather than which model to pick. P3 made the point, 💬 *"rather than a requirement that the agent be based on model X, it is that*

the agent use a model with certain reasoning performance, or a certain token usage." The choice is also driven by cost: teams match model capability to task difficulty rather than defaulting to the most powerful model, as P9 put it, 💬 *"if I have to do semantic classification, using [a top-tier model] is just burning money—I could use one that costs a third, maybe a tenth."* P23 described differentiating models by task across endpoints, reserving a stronger model for the critical step and an open-source one for the easier extractions. The technology is sometimes named and sometimes not, and P5 estimated that 💬 *"about half the time the name of the technology is known in the requirement."* (🧍 14 participants contributed to this).

E ***Socio-technical configuration of RE*** is about who takes part in RE process and who decides, and the set of roles grows with the project while communication is layered. P3 illustrated the layering, 💬 *"we never speak directly to the manager; we always go through our Scrum Master."* Authority usually rests with the client or management, as P3 noted, 💬 *"sometimes management hands down ideas you just have to add, that make no sense."* (🧍 21 participants contributed to this).

F ***Non-determinism and agent quality attributes*** underlies other conditions, because the agent is non-deterministic and behaves like a black box. P1 framed the difference, 💬 *"the process is fundamentally different… one of the biggest differences is that it is a black box, whereas traditional software is a white box."* This opacity pushes teams to add quality and control requirements, and P5 said, 💬 *"non-determinism is the main problem, so we describe some extra requirements on the constraints and the validation of these models."* (🧍 20 participants contributed to this).

G ***Behavioral evaluation and acceptance*** is how the team decides the agent is good enough, judging behavior with a test set and edge cases before a human signs off. P3 described the test set, 💬 *"we start from a test set, often generated by an LLM, and we measure performance for each prompt."* P4 added the edge cases, 💬 *"we identify the edge cases together and stress the agent on them; our acceptance criterion is that it at least passes the edge cases."* (🧍 23 participants contributed to this).

Other than the categories mentioned and described above, an anatomy of the process emerged (for which we adopted the vocabulary from software-process standards [47]–[49]). It distinguishes a *phase*, a major block of the process with its own goal; a *step*, the smallest unit of work within a phase; a *technique*, the method used to carry out a step; and an *artifact*, the output a step produces. We treat *maturity*, namely how mature the process and the client are, as a separate property rather than a level of this hierarchy. A second cross-cutting dimension—the requirement typology, i.e., the kinds of requirements practitioners write—is best introduced alongside the object that flows through the loop, and we therefore defer it to Section IV-B.

TABLE II
EXAMPLES OF LINKS FROM THE CATEGORIES TO THE THEORY.

	Category	Role in the theory	Tension
A	Requirements as behavioral/eval. specs	Object that flows through the loop	T4[d]
B	Iterative requirement discovery	Central category—opens (*scope*) and closes (*discover*) the loop	T4[d]
C	Prompt as requirement–implementation boundary	*Build* phase (the prompt artifact)	T1[a]
D	Capability-driven technology selection	*Build* phase (model and tools)	—
E	Socio-technical configuration of RE	Context condition	T3[c]
F	Non-determinism and agent quality attributes	Context condition	T2[b]
G	Behavioral evaluation and acceptance	*Evaluate* phase	—

[a] T1: requirement VS implementation, centered on the prompt.
[b] T2: control VS non-determinism.
[c] T3: client/business VS technical team.
[d] T4: fixed specification VS failure-driven requirement.

B. Behavioral Agent Requirement Discovery (BARD)

Following STGT, we worked in the *emergent* mode and reached the theory through theoretical structuring rather than a pre-defined template [15] (Table II). Moreover, in line with Ralph's guidelines [16], we developed a *process theory*—i.e., one that accounts for how an engineering entity changes over time, rather than linking static variables to an outcome as in variance theories. We name it **Behavioral Agent Requirement Discovery (BARD) Theory**, whose title captures our core finding that, phase after phase, teams discover the behavioral and evaluation criteria fixed specifications cannot set in advance. The process draws on two of the four motors of change of Van de Ven and Poole [50]. First, it is *teleological* (🎯), since a goal-directed team envisions an objective, acts, evaluates the output, and refines its goals from what it observes. Second, it is *dialectical* (⚡), since each move from one phase to the next is driven by two competing forces whose resolution establishes the state that opens the following phase.

The central category, ***iterative requirement discovery***, anchors the framework and shapes the loop at the top of Figure 3, which we present phase by phase, flagging the goal-directed activities (🎯) and the tensions (⚡). Within each *phase*, we identify the *steps*, the *techniques* used to carry them out, and the resulting *artifacts*, reporting this anatomy as it emerged from our data and avoiding decompositions where practitioners reported none.

Rather than a rigid 1:1 mirroring, the four process phases recombine the categories (Table II). Specifically, *scope* and *discover* open and close the primary execution loop. The *build* phase draws on prompt boundaries (C) and technology selection (D), while *evaluate* assesses behavioral acceptance (G). The requirement (A) acts as the core object flowing through this process, directly affecting the intermediate artifacts in the left layer of Figure 3. Finally, the entire loop is framed by the socio-technical conditions shown at the right of Figure 3: the

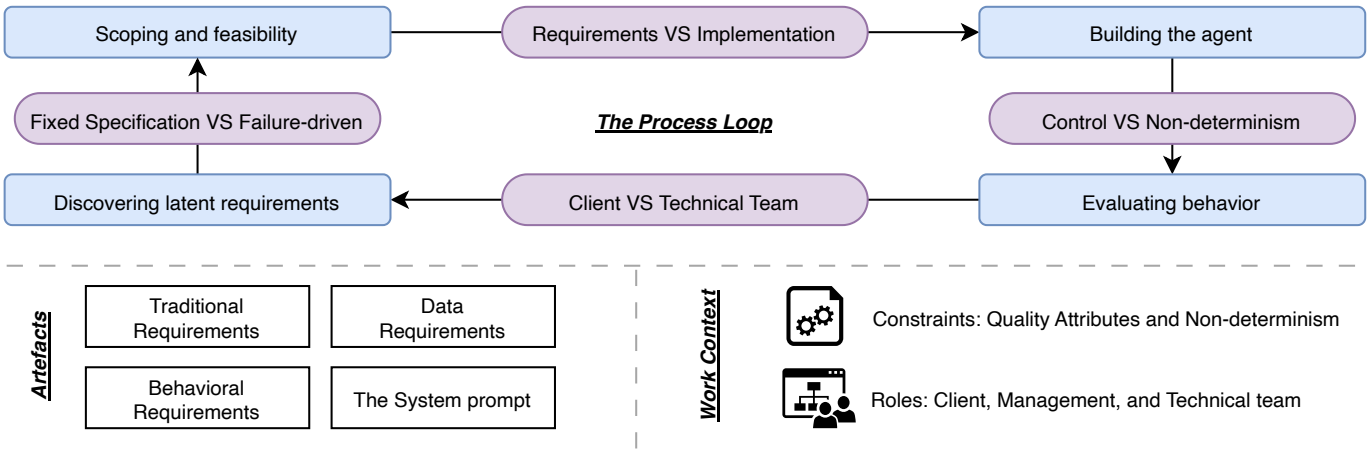


Fig. 3. Behavioral Agent Requirement Discovery—The BARD Substantive Process Theory. The teleological dimension of the theory (the process) is shown in blue, while the dialectical dimension (the forces driving the process forward) is shown in purple.

socio-technical configuration (E) and non-determinism (F).

The loop is a common pattern, though not the only one, and its depth depends on the client's maturity, the organization's type, and the team's formality. The loop is also not always iterative, since P1 described their own work as 🗨 *"a series of prototypes"* rather than incremental iteration. Conversely, as a boundary negative case, a small and well-scoped project allowed P17 to stabilize requirements up front, 🗨 *"we wrote a simple specification; the first drawn-out requirements were pretty spot on and decided how the project turned out."*

The object that flows through the loop is the requirement—from the category ***requirements as behavioral and evaluative specifications***—and it comes in several kinds, the requirement typology: *traditional* requirements, shared with non-AI software, such as functional, parametric, or interface requirements; *agent-specific* requirements, about what the agent may decide and do on its own, including the tools it may use; *chain-of-thought* requirements, about the depth and form of the model's reasoning; *prompt-specific* requirements, carried by the system prompt, such as the agent's persona and behavioral constraints; *interaction-based* requirements, about the interaction flow with the user; and *data* requirements, about the data the agent needs and its quality. P1 named two of these kinds directly, 🗨 *"defining the level of chain of thought, and then the quality of the prompt and so the quality of the answer… all degrees of freedom over the quality of the output."* Moreover, P13 said: 🗨 *"we had to build some tools that the agent had to use… and write a prompt about its skills, its tasks, and the tools."* What sets an agent requirement apart is that it aims at behavior—not at an exact output—and much of it remains implicit. Its most concrete carrier along the loop is the system prompt where prompt-specific requirements are encoded at *build* and judged at *evaluate*. Across the loop, the requirement changes form, starting as an up-front specification and ending as a set of behavioral and evaluation criteria, and each phase reshapes it; P20 noted: 🗨 *"we were given a high-level requirement and created a design off the back of this… our scope was what changed most."*

- **Scoping and feasibility.** The loop starts from a vague need rather than a specification, and the early work can still look like classic RE. P5 described the opening, 🗨 *"first there is a preliminary contact with the client, where the needs are clarified, and we understand whether we can meet their request."* The first agent-specific move is to decide what genuinely needs an agent and what does not, a judgment we call AI/traditional requirement splitting and that P6 records as a feasibility and splitting matrix. How deep this step goes depends on the client's maturity, which P6 made explicit, 🗨 *"we classified levels of client maturity, from using an LLM for personal productivity, to RAG over your data, to agentic AI, up to a control plane with autonomous agents."* The output is rarely a written spec but often a Minimum Viable Product (MVP), as P6 explained, 🗨 *"the client describes a pain point and we produce a MVP on the spot, so they grasp the potential of the platform."* What moves the team into building is the first tension, ϟ requirement versus implementation: the scoped needs must now be encoded, and the prompt that will carry much of them is written like implementation yet fixes what the agent must do—P14 saw 🗨 *"a requirements decision"* while P12 saw 🗨 *"a technical/implementation tweak."* The resolution is to treat the prompt as a requirements decision taken within a technical discussion, and the team enters the next phase with requirements to encode.
- **Building the agent.** The team builds the agent, starting from the problem domain before moving to the prompt, and most of the effort goes into prompt and context engineering. P3 said, 🗨 *"80% of the work is at the level of prompting, but there is always that small part you must refine more and more to reach decent performance."* The main artifact is the system prompt, where the requirements decided in the previous phase are now encoded. Notably, participant follow-ups emphasized a strong alignment here: the system prompt often completely bypasses separate up-front specifications,

serving directly as the primary requirements document itself. The team also selects the model and tools, where the requirement names what the model must do, as P1 recalled, 🗨 *"only the characteristics we expected: a certain speed, and the ability to run locally."* What moves the team onward is the second tension, ϟ control versus non-determinism. This tension involves a trade-off between strict deterministic control (which limits the agent's capabilities) and its non-deterministic flexibility to achieve required performance. P1 observed that agentic software is selected precisely because its execution is 🗨 *"closer to human reasoning than to a set of finite and deterministic steps,"* meaning that over-constraining the model neutralizes its primary utility. The resolution of this tension requires accepting a baseline of stochastic independence to preserve the agent's generative utility, while calibrating operational boundaries based on task-specific risks. This decision effectively drives the transition into the next phase, where they proceed to evaluate the resulting behavior.

- **Evaluating behavior.** Because the output is non-deterministic, the team cannot check a single fixed result but must judge behavior instead. The behavior is judged against criteria using test sets, checklists, edge cases, and repeated runs (P3, P4). As P10 put it: 🗨 *"there will always be outliers… you are trying to reach a threshold that is acceptable for the client."* A human then signs off, and the third tension moves the team into the last phase, ϟ client versus technical team: the people who build the agent are not the ones who accept it, and the resolution gives the client the last word, as P10 said, 🗨 *"the client has the final say after we give them all the information and stats."* What the sign-off rejects, or what the repeated runs expose, is handed over to the next phase.
- **Discovering latent requirements.** Evaluation does more than pass or fail, because it can also produce new requirements. A failure can reveal a requirement that no one had written down, as P11 described, 🗨 *"we reviewed the failure cases and added a requirement to avoid ungrounded confidence, and the product manager delayed release."* A severe failure can force this directly, as P19 recalled, 🗨 *"the log-analysis agent took its 'analyst' persona too literally and began writing scripts to scan external IPs… I proposed a hard rewrite of its system prompt."* This highlights how agent failures act as an elicitation trigger in some cases. The same process that, in traditional software, uncovers coding bugs here exposes unstated behavioral boundaries and missing semantic constraints in operational edge cases. This drives the fourth tension that closes the loop, i.e., ϟ fixed specification versus failure-driven requirement, where static specs cannot anticipate what the running agents reveal. P11 argued that 🗨 *"requirements are better defined as failure modes, like overconfidence, not static specs;"* P14 also noted that 🗨 *"the requirements definition phase should be synonymous with defining the evaluation criteria."* The resolution follows: requirements are re-expressed as evaluation criteria for the next scope cycle.

The process above is further informed by two conditions that shape the surrounding context in which the team has to develop the agent system. The first is the ***socio-technical configuration of RE***, that is, who takes part and who decides; the set of roles grows with the project, and authority typically rests with the client or with management, which is what charges the ϟ client versus technical team tension above. Feedback from participants indicated that the final authority often also shifts to compliance and regulatory constraints (e.g., the EU AI Act), which can block or redesign requirements before the behavioral evaluation is even carried out. The second condition is ***non-determinism and agent quality attributes***, and it runs deeper than the first, because it is the reason the loop exists at all—since the agent is stochastic, the team cannot fix and verify a requirement a single time, but must keep judging behavior across runs. Non-determinism also pushes to the foreground a class of quality requirements that stays in the background of traditional software, such as guardrails and fallback, factual correctness, latency, and the cost of tokens. This condition therefore presses on every phase, from the splitting of feasible work to the acceptance check. P3 stressed the need for safety nets, 🗨 *"a whole series of fallback mechanisms that, in my view, must always be implemented."*

V. Discussion

Our theory holds that engineering requirements for GenAI agents is a process of progressive discovery—i.e., the requirement is not fixed at the start but takes shape as the team builds the agent, observes its behavior, and revises what it asks for, phase after phase.

BARD suggests that three distinctions classical RE keeps separate blur once the system is an agent: (i) requirement versus evaluation criterion, as teams stop stating an exact output and start stating the behavior they will accept; (ii) requirement versus implementation, as the system prompt carries both at once; and (iii) eliciting requirements from stakeholders versus discovering them from behavior, as it is often a failure of the running agent that reveals the requirement.

The process, therefore, centers on evaluation rather than on an upfront specification.Two conditions explain why this is more than a local practice—namely, the non-determinism of the agent (a black-box reasoning core that the team cannot specify and verify once) and a socio-technical configuration in which the people who run the loop are rarely the ones who accept its result.

In the following, we evaluate BARD against the criteria that Hoda [15] sets for mature theories resulting from full STGT studies, i.e., novelty, usefulness, parsimony, and modifiability.[3]

A. Discussing Novelty

To the best of our knowledge, BARD is the first substantive theory of the RE process for GenAI agents. Novelty could be assumed as a default once the credibility and rigor of the

[3]Two further criteria, credibility and rigor, evaluate the application of the method rather than its outcome [15]. They are based on the process described in detail in Section III; for reasons of space, we do not repeat here.

application are established [15]; nevertheless, since novelty is best scrutinized where a theory resembles existing models, we compare BARD with similar theories. Following Ralph [16], we compare BARD with high-level theories rather than operational RE methods; a process theory accounts for *what* happens and *why*, whereas methods prescribe *how* to proceed.

The closest body of work is RE for AI- and ML-based systems, surveyed by Habiba et al. [8] and mapped by Villamizar et al. [9], with early agendas from Vogelsang and Borg [10] and Belani et al. [11]. This literature reports new quality requirements—e.g., explainability, data quality, and the non-functional ones documented by Habibullah et al. [51] and De Martino et al. [52]. They both suggest that learning from data weakens up-front specification, which Kuwajima et al. [53] frame as a missing requirements specification. BARD advances these works in two ways. First, it moves the unit of analysis from the requirement to the process that produces it—how a requirement forms, changes, and stabilizes phase after phase. Second, it takes the agent as the object, where tool use, the system prompt, and the depth of reasoning create requirement types that ML-centered accounts do not consider.

A second family of work moves beyond fixed up-front specifications. Goal-oriented and agent-oriented RE approaches, such as KAOS and Tropos [27], [28], discussed in Section II-C, keep requirements open to refinement. However, they assume a rational agent whose reasoning logic is explicitly engineered. In contrast, the agent in BARD is stochastic, and its behavior is shaped indirectly through prompts.

In the context of requirements at runtime for self-adaptive systems, Sawyer et al. [54] describe this agenda as requirements reflection, where requirements remain runtime entities that the system can reason about. However, the source of uncertainty is different. In self-adaptive systems, uncertainty concerns the environment, such as changing load or resources, and the system adapts through an automated feedback loop. In BARD, uncertainty concerns the behavioral non-determinism of the artifact itself, and the human team discovers and revises requirements during development rather than at runtime. Thus, the two lines of work describe different phenomena, and BARD is not simply a special case of requirements reflection.

Lastly, the Twin Peaks model of Nuseibeh [55] says that requirements and architecture are intertwined and co-evolve rather than being settled in sequence. BARD moves the classic requirements-architecture pairing toward a requirements-implementation pairing. This system prompt mediates this—an artifact written like code that specifies agent behavior, merging requirements and implementation into a single object.

B. Discussing Usefulness

Hoda [15] argues that a mature theory should not only resonate with practice, but also provide actionable insights for practitioners and researchers. At the same time, relevant grounded findings may appear unsurprising to experienced readers [15]. For example, the fact that teams iterate is not new. What BARD explains is more specific: how iteration changes the requirement itself, turning an initial specification into behavioral and evaluation criteria, and which tensions drive these transitions. In the following, we discuss the theory's applications and implications for practitioners and researchers.

1) Usefulness for Practitioners: We identify four main implications for practitioners:

- BARD gives teams a process map and a shared vocabulary. Practitioners can use it to understand where they are in the loop, name the tension they are facing, and treat the discovery of latent requirements as an expected part of development. For example, teams can plan evaluation cycles rather than treating these discoveries as scope creep.
- BARD also highlights the system prompt as a requirements artifact, not only as implementation code. This suggests that teams should version prompts, review them with stakeholders, and trace requirements decisions into them.
- Since acceptance is behavioral, teams can agree with clients early that sign-off will depend on behavior across repeated runs, test sets, and edge cases, rather than on exact outputs. This can reduce the friction captured by the client versus technical team tension.
- Finally, BARD suggests that teams should make AI/traditional requirement splitting an explicit early activity. The depth of this activity can be adapted to the client's maturity and ability to reason about GenAI Agents.

2) Usefulness for Researchers: We identify three main implications for researchers:

- BARD provides a process frame for organizing narrower findings on GenAI Agent development. For example, studies on quality requirements, prompt design, or evaluation methods can be related to the phase and tension they address.
- As a process theory, BARD also offers propositions that future studies can test, following Ralph's recommendations [16]. For example, future work could examine whether the loop becomes less visible when its underlying conditions weaken, and whether it remains stable when behavioral non-determinism and external acceptance authority are present.
- Finally, BARD identifies concrete units of observation for future empirical work, such as system prompts, splitting matrices, and evaluation suites. Studying these artifacts would help corroborate the theory and address the single-channel limitation discussed in Section VI.

C. Discussing Parsimony and Modifiability

The last two criteria can be argued briefly.

Parsimony asks that a theory explain a complex phenomenon in a compact and elegant form [15]. BARD compresses into a single statement and a single figure (Figure 3): RE for GenAI agents proceeds as a four-phase loop, driven by four tensions under two conditions, in which the requirement moves from a fixed specification toward behavioral and evaluation criteria. Section IV unpacks the richness behind this compact form. Furthermore, we returned the theory to the practitioners we interviewed, and they confirmed the results we obtained as well as reporting that it is understandable.

Modifiability asks that a theory absorb new evidence and new contexts rather than be falsified by them [15]. BARD

is modifiable by construction, since its unit of analysis is the process rather than any specific technology. As argued in Section VI, the surface details will date faster than the loop, and data from new contexts—e.g., regulated domains or large organizations with formal requirements governance—can extend the theory where we expect it to thin out.

VI. Limitations

This section discusses the limitations of our study.[4]

The first threat concerns the *composition of our sample*. The SIGSOFT Empirical Standards [37] warn against small homogeneous samples that create the illusion of convergence and theoretical saturation. Our sample is uneven along two visible axes, gender and geography, as reported in Table I. We took the mitigations described in Section III-A: active diversity-seeking through theoretical sampling across role, application domain, organizational type, and seniority, convenience sampling complemented by snowballing to reach beyond our direct network, and a substantive- rather than formal-saturation framing, so that the spread along these dimensions counterbalances the gender and geographic skew. Future larger studies will be conducted to robustly extend our results over the original context. Nevertheless, as already noted, our sample is coherent with general developer demographics [42].

A second threat is *researcher bias*. We treat reflexivity as an ongoing obligation, and we detail in Section III-D our positions and the steps to keep any one of them from dominating the coding. Those steps have a limit, namely, our checks on interpretation stayed team-internal—e.g., rotating coding groups and consensus building—rather than passing through an external coder audit, so a blind spot shared across SE researchers could survive the analysis. Nevertheless, it is worth noting that we returned the theory to the practitioners we interviewed, and they confirmed our results (Section III-E); we consider this solid evidence that this risk has been mitigated.

A third threat is our *single evidence channel*. As stated in Section III-E, interviews are our only source, so we rely on practitioner self-reports without observing practice or examining artifacts such as system prompts, tool manifests, and evaluation suites. Indeed, self-reports can smooth over what teams actually do, and corroborating the theory against such evidence is a concrete direction for follow-up work.

Our theory is also *bound in time*. GenAI agent technology is moving fast, and the resulting practices reflect the tools and norms available in the submission window. As the technical substrate changes, some surface details—e.g., the available models, the tool interfaces, and the prompting patterns—will date faster than the process we describe, whose unit of analysis is how requirements form and stabilize rather than any specific technology. Nevertheless, the theory captures a socio-technical phenomenon that is undoubtedly more robust than findings focused solely on technical aspects, serving as a foundation to easily update our future understanding of the phenomenon.

[4]According to the ACM/SIGSOFT Empirical Standards [37], we evaluate trustworthiness through qualitative criteria (credibility, dependability, transferability) rather than metrics such as internal validity or reliability.

Finally, we state the *transferability* of the theory, as recommended for grounded-theory studies [15]. Our participants were practitioners involved in the development of GenAI Agent systems, mostly working in small-to-medium teams in consultancy/B2B or internal-product settings. Therefore, our claim is one of transferability to contexts where requirements for agent systems are engineered by human teams that iteratively discover, refine, and validate agent behavior during development, rather than fully specifying it up front.

At the same time, the theory is likely to thin out where these conditions change substantially. This may include heavily regulated or safety-critical domains, where requirements are subject to stronger up-front specification and compliance constraints, as well as very large organizations with formal requirements governance structures. Given the demographic composition of our sample, further investigations are needed to assess the theory's transferability across more diverse geographic, organizational, and domain contexts.

RQ—*How do practitioners engineer requirements for software systems built around GenAI Agents?*

Requirements for GenAI agents are not fixed up front but *discovered*. Phase after phase, the team scopes the need, builds the agent, observes its behavior, and revises what it asks for, so the requirement shifts from a fixed specification toward behavioral and evaluation criteria. This shift is driven by four recurring tensions and framed by two conditions—namely, the agent's non-determinism and who holds authority over the loop.

VII. Conclusion

In this paper, we propose *Behavioral Agent Requirement Discovery* (BARD), a substantive theory holding that requirements for GenAI agent systems are not fixed up front but discovered as the team scopes the need, builds the agent, observes its behavior, and revises what it asks for. BARD can support both researchers and practitioners: the former obtain a process frame in which narrower findings—e.g., on quality requirements, prompt design, or evaluation methods—can be positioned; the latter, a vocabulary and a set of phases to organize work that is often ad hoc.

As future work, we plan to develop quantitative instruments to support measuring developers' well-being and AI-readiness during all the phases of our process. Moreover, a natural next step is to extend the theory to other steps of the development process—e.g., design and testing—aiming at developing a comprehensive contribution.

Data Availability

All data are available at the following link: https://figshare.com/s/582e12f660425f29c9cb

Acknowledgment

The authors would like to thank the participants who participated in the interviews and provided the data.

REFERENCES

[1] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges," *Information Fusion*, p. 103599, 2025.

[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[3] S. Lambiase, G. Catolino, F. Palomba, and F. Ferrucci, "Motivations, challenges, best practices, and benefits for bots and conversational agents in software engineering: A multivocal literature review," *ACM Comput. Surv.*, vol. 57, no. 4, Dec. 2024. [Online]. Available: https://doi.org/10.1145/3704806

[4] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of chatgpt in code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–28, 2025.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.

[7] A. E. Hassan, D. Lin, G. K. Rajbahadur, K. Gallaba, F. R. Cogo, B. Chen, H. Zhang, K. Thangarajah, G. Oliva, J. Lin *et al.*, "Rethinking software engineering in the era of foundation models: A curated catalogue of challenges in the development of trustworthy fmware," in *Companion Proceedings of the 32nd ACM international conference on the foundations of software engineering*, 2024, pp. 294–305.

[8] U.-e. Habiba, M. Haug, J. Bogner, and S. Wagner, "How mature is requirements engineering for AI-based systems? a systematic mapping study on practices, challenges, and future research directions," *Requirements Engineering*, 2024.

[9] H. Villamizar, T. Escovedo, and M. Kalinowski, "Requirements engineering for machine learning: A systematic mapping study," in *2021 47th Euromicro conference on software engineering and advanced applications (SEAA)*. IEEE, 2021, pp. 29–36.

[10] A. Vogelsang and M. Borg, "Requirements engineering for machine learning: Perspectives from data scientists," in *2019 IEEE 27th international requirements engineering conference workshops (REW)*. IEEE, 2019, pp. 245–251.

[11] H. Belani, M. Vukovic, and Ž. Car, "Requirements engineering challenges in building ai-based complex systems," in *2019 IEEE 27th international requirements engineering conference workshops (REW)*. IEEE, 2019, pp. 252–255.

[12] F. Dalpiaz and N. Niu, "Requirements engineering in the days of artificial intelligence," *IEEE Software*, vol. 37, no. 4, pp. 7–10, 2020.

[13] Z. Wan, X. Xia, D. Lo, and G. C. Murphy, "How does machine learning change software development practices?" *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1857–1871, 2021.

[14] P. Ralph, M. Chiasson, and H. Kelley, "Social theory for software engineering research," in *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '16. New York, NY, USA: Association for Computing Machinery, 2016.

[15] R. Hoda, "Socio-technical grounded theory for software engineering," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3808–3832, 2021.

[16] P. Ralph, "Toward methodological guidelines for process theories and taxonomies in software engineering," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 712–735, 2018.

[17] B. G. Glaser and A. L. Strauss, *Discovery of Grounded Theory: Strategies for Qualitative Research*. Routledge, 2017.

[18] S. Lambiase, G. Catolino, F. Palomba, F. Ferrucci, and D. Russo, "Exploring individual factors in the adoption of llms for specific software engineering purposes," 2026. [Online]. Available: https://arxiv.org/abs/2504.02553

[19] A. Della Porta, S. Lambiase, and F. Palomba, "Do prompt patterns affect code quality? a first empirical assessment of chatgpt-generated code," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, 2025, pp. 181–192.

[20] S. Lambiase, G. Catolino, F. Palomba, F. Ferrucci, and D. Russo, "Investigating the role of cultural values in adopting large language models for software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 1, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3725529

[21] R. Choudhuri, B. Trinkenreich, R. Pandita, E. Kalliamvakou, I. Steinmacher, M. Gerosa, C. Sanchez, and A. Sarma, "What guides our choices? modeling developers' trust and behavioral intentions towards genai," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1691–1703.

[22] A. D. Porta, G. Recupito, S. Lambiase, D. Di Nucci, and F. Palomba, "Unlocking code simplicity: The role of prompt patterns in managing llm code complexity," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, 2025, pp. 140–143.

[23] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[24] A. Della Porta, G. Voria, A. Abbate, R. Sulipano, S. Lambiase, G. Catolino, and F. Palomba, "Toward measuring prompt quality: A preliminary investigation on prompt smells," in *Proceedings of the SQA4AI Workshop, co-located with SANER 2026*, 2026.

[25] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, "Software engineering for AI-based systems: A survey," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 2, pp. 1–59, 2022.

[26] K. Ahmad, M. Abdelrazek, C. Arora, M. Bano, and J. Grundy, "Requirements engineering for artificial intelligence systems: A systematic mapping study," *Information and Software Technology*, vol. 158, p. 107176, 2023.

[27] A. van Lamsweerde, "Goal-oriented requirements engineering: A guided tour," in *Proceedings of the Fifth IEEE International Symposium on Requirements Engineering (RE'01)*. IEEE, 2001, pp. 249–262.

[28] P. Bresciani, A. Perini, P. Giorgini, F. Giunchiglia, and J. Mylopoulos, "Tropos: An agent-oriented software development methodology," *Autonomous Agents and Multi-Agent Systems*, 2004.

[29] J. Horkoff, F. B. Aydemir, E. Cardoso, T. Li, A. Maté, E. Paja, M. Salnitri, L. Piras, J. Mylopoulos, and P. Giorgini, "Goal-oriented requirements engineering: An extended systematic mapping study," *Requirements Engineering*, vol. 24, no. 2, pp. 133–160, 2019.

[30] R. Hoda, "Qualitative research with socio-technical grounded theory a practical guide to qualitative data analysis and theory development in the digital world," *Innovations*, 2025.

[31] S. Lambiase, G. Catolino, B. D. Piana, F. Ferrucci, and F. Palomba, "Dealing with cultural dispersion: A novel theoretical framework for software engineering research and practice," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*, 2024, pp. 74–84.

[32] K. Madampe, R. Hoda, and J. Grundy, "The emotional roller coaster of responding to requirements changes in software engineering," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1171–1187, 2022.

[33] ——, "Supporting emotional intelligence, productivity and team goals while handling software requirements changes," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 1–38, 2024.

[34] H. Gunatilake, J. Grundy, R. Hoda, and I. Mueller, "The role of empathy in software engineering–a socio-technical grounded theory," *arXiv preprint arXiv:2504.13002*, 2025.

[35] M. Swillus, R. Hoda, and A. Zaidman, "On the emergence of testing strategies: A socio-technical grounded theory," *Empirical Software Engineering*, vol. 31, no. 5, p. 147, 2026.

[36] V. De Martino, S. Lambiase, F. Pecorelli, W.-J. van den Heuvel, F. Ferrucci, and F. Palomba, "Sustainability of machine learning-enabled systems: The machine learning practitioner's perspective," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 7, Jun. 2026. [Online]. Available: https://doi.org/10.1145/3777553

[37] P. Ralph, N. b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri *et al.*, "Empirical standards for software engineering research," *arXiv preprint arXiv:2010.03525*, 2020.

[38] B. Glaser, "Theoretical sensitivity," *Advances in the methodology of grounded theory*, 1978.

[39] A. Strauss and J. Corbin, *Basics of qualitative research techniques*. Thousand oaks, CA: Sage publications, 1998.

[40] K. Charmaz, *Constructing Grounded Theory*, 2nd ed. London: SAGE Publications, 2014.

[41] S. Baltes and P. Ralph, "Sampling in software engineering research: A critical review and guidelines," *Empirical Software Engineering*, vol. 27, no. 4, p. 94, 2022.
[42] Stack Overflow, "Stack overflow annual developer survey 2025," https://survey.stackoverflow.co/2025/, 2025, accessed: 2026-06-27.
[43] B. Bruegge and A. H. Dutoit, *Object oriented software engineering: Using UML patterns and Java*. Prentice Hall, 2010.
[44] A. Cannavale, A. Della Porta, G. Voria, G. Recupito, M. T. Baldassarre, G. Catolino, F. Palomba, A. De Lucia, and S. Lambiase, "Replication package of "Engineering Requirements for Generative AI Agents: The BARD Process Theory"," https://figshare.com/s/582e12f660425f29c9cb, 2026.
[45] "Miro dashboard," https://miro.com/app/dashboard/.
[46] K.-J. Stol, P. Ralph, and B. Fitzgerald, "Grounded theory in software engineering research: A critical review and guidelines," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 2016, pp. 120–131.
[47] Object Management Group, *Software & Systems Process Engineering Meta-Model Specification (SPEM), Version 2.0*, Object Management Group, 2008. [Online]. Available: https://www.omg.org/spec/SPEM/2.0/
[48] ISO/IEC/IEEE, *ISO/IEC/IEEE 12207:2017 — Systems and Software Engineering — Software Life Cycle Processes*, International Organization for Standardization, Geneva, Switzerland, 2017.
[49] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Heidelberg: Springer, 2010.
[50] A. H. Van de Ven and M. S. Poole, "Explaining development and change in organizations," *Academy of management review*, vol. 20, no. 3, pp. 510–540, 1995.
[51] K. M. Habibullah *et al.*, "Non-functional requirements for machine learning: Understanding current use and challenges among practitioners," *Requirements Engineering*, 2023.
[52] V. De Martino and F. Palomba, "Classification and challenges of non-functional requirements in ml-enabled systems: A systematic literature review," *Information and Software Technology*, vol. 181, 2025.
[53] H. Kuwajima *et al.*, "Engineering problems in machine learning systems," *Machine Learning*, 2019.
[54] P. Sawyer, N. Bencomo, J. Whittle, E. Letier, and A. Finkelstein, "Requirements-aware systems: A research agenda for re for self-adaptive systems," in *2010 18th IEEE International Requirements Engineering Conference*. IEEE, 2010, pp. 95–103.
[55] B. Nuseibeh, "Weaving together requirements and architectures," *Computer*, vol. 34, no. 3, pp. 115–119, 2001.